HDOC Day-54

Write a C program that reads the size and elements of a variable-length integer array, arr[], and an integer k such that 0 < k < size. Pass the array to a function named max_subarray_sum() using its address. Use pointers to pass the array between the calling function and max_subarray_sum(). The function max_subarray_sum() must:

- Identify all contiguous subarrays of size k in arr[].
- Calculate the sum of the elements in each subarray.
- Find and display the maximum subarray sum.
- Display -1 if k is invalid.

Example 1:

Array: 2 1 5 1 3 2
k: 3

Output:

Maximum subarray sum: 9

[Explanation: The subarrays of size 3 are: 2+1+5=8, 1+5+1=7, 5+1+3=9, 1+3+2=6]

Example 2:

Array: 2 3 4 1 5
k: 2

Output:

Maximum subarray sum: 7

Example 3:

Array: 1 2 3
k: 4

Output:

-1

Algorithm

1. Start.
2. Read the size n of the array.
3. Read n integer elements into arr[].
4. Read the value of k.
5. Pass the address of arr[], n, and k to max_subarray_sum().
6. Inside max_subarray_sum():
    - If k <= 0 or k > n, display -1 and return.
    - Calculate the sum of the first k elements.
    - Store this sum as maxSum.
    - Move the window one position at a time:
        - Subtract the element leaving the window.
        - Add the new element entering the window.
        - If the new sum is greater than maxSum, update maxSum.
7. Display the maximum subarray sum.
8. Stop.

C Program:

```
#include <stdio.h>

void max_subarray_sum(int *arr, int size, int k)

{

  int i, sum = 0, maxSum;


  // Check whether k is valid

  if (k <= 0 || k > size)

  {

    printf("-1\n");

    return;

  }
```

```
    // Sum of the first subarray of size k
    for (i = 0; i < k; i++)
    {
        sum += *(arr + i);
    }

    maxSum = sum;

    // Find sums of the remaining subarrays
    for (i = k; i < size; i++)
    {
        sum = sum - *(arr + i - k) + *(arr + i);

        if (sum > maxSum)
        {
            maxSum = sum;
        }
    }

    printf("Maximum subarray sum: %d\n", maxSum);
}

int main()
{
    int size, k, i;

    printf("Enter size: ");
    scanf("%d", &size);
```

```
    int arr[size];

    printf("Enter array elements: ");
    for (i = 0; i < size; i++)
    {
        scanf("%d", &arr[i]);
    }

    printf("Enter k: ");
    scanf("%d", &k);

    // Array is passed using its address
    max_subarray_sum(arr, size, k);

    return 0;
}
```

Test Case Table:

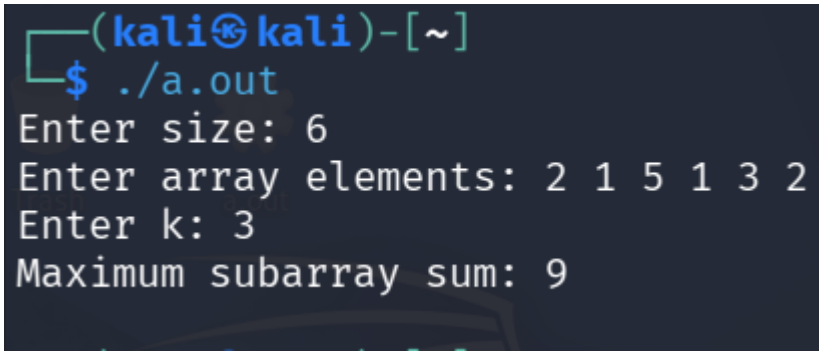
Test Case	Array	Size	k	Subarray Sums	Expected Output
1	2 1 5 2 3 2	6	3	8 , 7 , 9 , 6	Maximum subarray sum: 9
2	2 3 4 1 5	5	2	5 , 7 , 5 , 6	Maximum subarray sum:7
3	1 2 3	3	4	Invalid	-1
4	1 2 3 4 5	5	3	6 , 9 , 12	Maximum subarray sum: 12
5	-5 -2 -3 -1	4	2	-7 , -5 , -4	Maximum subarray sum: -4

6	5 10 15 20	4	1	5 , 10 , 15 , 20	Maximum subarray sum: 20
7	1 2 3	3	0	Invalid	-1

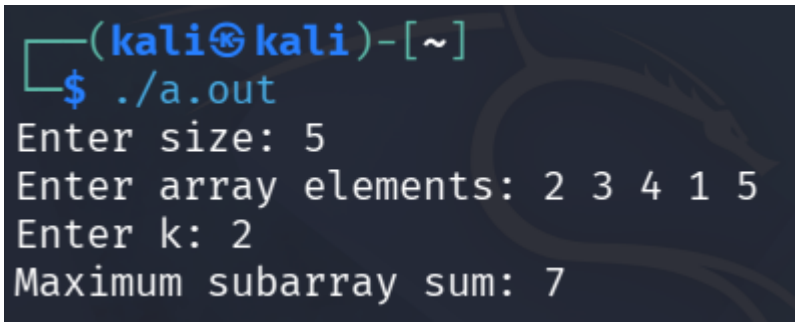
Compilation and Execution:

The following program has been compiled and executed successfully.

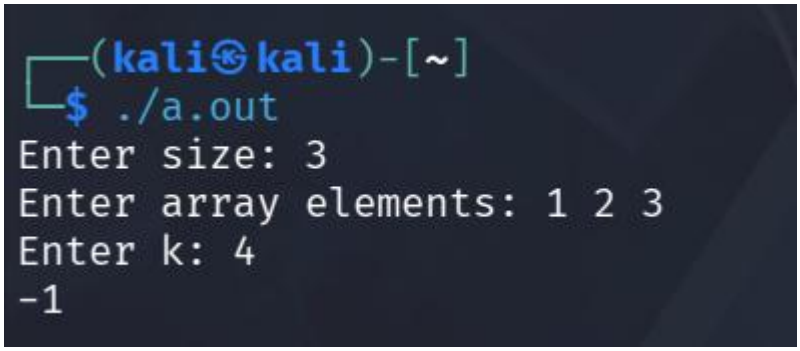
Test Case 1:



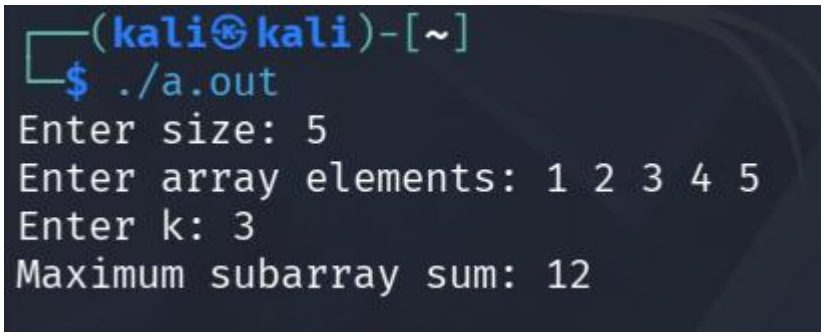
Test Case 2:



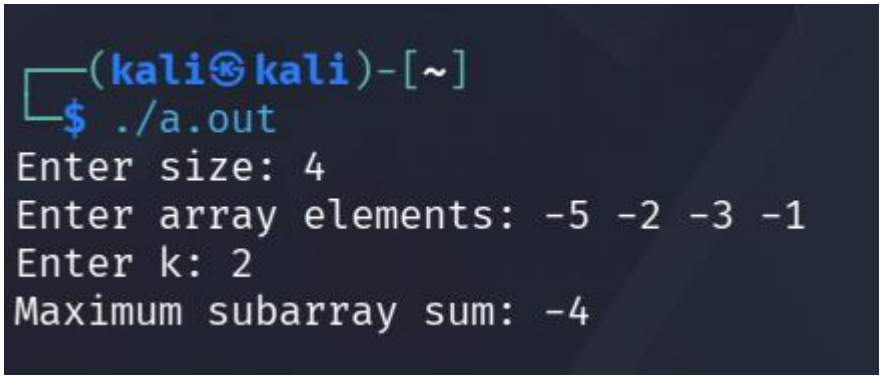
Test Case 3:



Test Case 4:



Test Case 5:



Test Case 6:

